Computer Science 111
Introduction to Computer Science I

Boston University, Spring 2026

The slides in this book are based in part on notes from the CS-for-All curriculum developed at Harvey Mudd College.

Introduction to Computer Science I

Course Overview

Computer Science 111
Boston University

Welcome to CS 111!

Computer science is not so much the science of computers as it is the science of solving problems using computers.

Eric Roberts

- This course covers:
  - the process of developing algorithms to solve problems
  - the process of developing computer programs to express those algorithms
  - other topics from computer science and its applications

Computer Science and Programming

- There are many different fields within CS, including:
  - software systems
  - computer architecture
  - networking
  - programming languages, compilers, etc.
  - theory
  - AI
- Experts in many of these fields don't do much programming!
- *However, learning to program will help you to develop ways of thinking and solving problems used in all fields of CS.*

A Breadth-Based Introduction

- Five major units:
  - weeks 0-4: computational problem solving and "functional" programming
  - weeks 4-6: a look "under the hood" (digital logic, circuits, etc.)
  - weeks 6-8: imperative programming
  - weeks 8-11: object-oriented programming
  - weeks 12-end: topics from CS theory
- In addition, short articles on other CS-related topics.
- Main goals:
  - to develop your computational problem-solving skills
    - including, but not limited to, coding skills
  - to give you a sense of the richness of computer science

A *Rigorous* Introduction

- Intended for:
  - CS, math, and physical science concentrators
  - others who want a rigorous introduction
  - no programming background required,
    but can benefit people with prior background
- Allow for 10-15 hours of work per week
  - start work early!
- Other alternatives include:
  - CS 103: the Internet and web programming
  - CS 108: programming for non-CS majors
  - DS 100: programming, data modeling and visualization
  - for more info:
    https://www.bu.edu/cs/undergraduate/courses

Course Materials

- **Required**: *The CS 111 Coursepack*
  - use it during pre-lecture and lecture – need to fill in the blanks!
  - PDF version is available on Blackboard
  - recommended: get it printed
    - one option: FedEx Office (Cummington & Comm Ave)
    - to order, follow the instructions we'll give you in Lab 0
- **Required** in-class software: Top Hat Pro platform
  - used for pre-lecture quizzes and in-lecture exercises
  - also used periodically for location-based attendance
  - create your account and purchase a subscription ASAP
- **Optional** textbook: *CS for All*
  by Alvarado, Dodds, Kuenning, and Libeskind-Hadas

Traditional Lecture Classes

- The instructor summarizes what you need to know.
- Readings are assigned but may not actually be done!
- Dates back to before the printing press.

- Many technological developments since then!

Limitations of the Traditional Approach

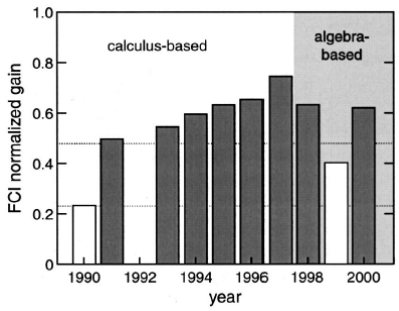
- You get little or no immediate feedback.
- Research shows that little is learned from passive listening.
  - need to actively engage with the material
- Homework provides active engagement, but in-class engagement provides added benefits.

Lectures in this Class

- Based on an approach called *peer instruction*.
  - developed by Eric Mazur at Harvard
- Basic process:
  1. Question posed (possibly after a short intro)
  2. Solo vote on Top Hat (no discussion yet)
  3. Small-group discussions (in teams of 3)
     - explain your thinking to each other
     - come to a consensus
  4. Group vote on Top Hat
     - each person in the group should enter the same answer
  5. Class-wide discussion

Benefits of Peer Instruction

- It promotes active engagement.
- You get immediate feedback about your understanding.
- I get immediate feedback about your understanding!
- It promotes increased learning.
  - explaining concepts to others benefits you!



traditional instruction

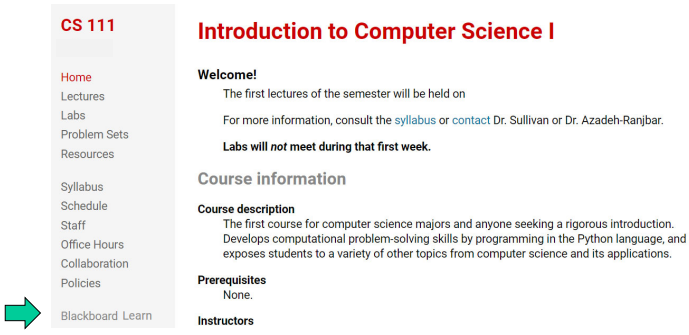
peer Instruction

Crouch, C., Mazur, E. Peer Instruction: Ten years of experience and results.

Preparing for Lecture

- Short video(s) and/or readings
  - fill in the blanks as you watch the videos!
- Short online reading quiz on Top Hat
  - complete **by 11 a.m.** of the day of lecture (unless noted otherwise)
  - won't typically be graded for correctness
  - your work should show that you've prepared for lecture
  - **no late submissions accepted**
- Preparing for lecture is essential!
  - gets you ready for the lecture questions and discussions
  - we won't cover everything in lecture

Course Website
www.cs.bu.edu/courses/cs111



- *not* the same as the Blackboard site for the course
- use Blackboard to access info. on:
  - the pre-lecture videos/readings
  - the pre-lecture quizzes

  } posted by 36 hours before lecture
  - list of pages covered in lecture

Labs

- Will help you prepare for and get started on the assignments
- Will also reinforce essential skills
- **ASAP: Complete Lab 0** (on the course website)
  - short tasks to prepare you for the semester

Assignments

- Weekly problem sets
  - most have two parts:
    - part I due by 11:59 p.m. on Thursday
    - part II due by 11:59 p.m. on Sunday
- Final project (worth 1.5 times an ordinary assignment)
- Can submit up to 24 hours late with a 10% penalty.
- No submissions accepted after 24 hours.

Collaboration

- Two types of homework problems:
  - individual-only: must complete on your own
  - pair-optional: can complete alone or with one other student
- For both types of problems:
  - may discuss the main ideas with others
  - may ***not* view** another student/pair's work
  - may ***not* show** your work to another student/pair
  - don't give a student unmonitored access to your laptop
  - don't consult solutions in books or online
  - **don't use tools that automate coding/problem-solving**
  - don't post your work where others can view it
- ***Students who engage in misconduct can face serious repercussions (see the syllabus).***

Grading

1. Weekly problem sets + final project (25%)
2. Exams
   - two midterms (30%) – **Wed nights 6:30-7:45**; no makeups!
   - final exam (35%)
     - can replace lowest problem set and lowest midterm
     - **wait until you hear its dates/times from me; make sure you're available for the entire exam period!**
3. Participation (10%)

To pass the course, you must have a passing PS average and a passing average across the three exams.

Participation

- Full credit if you:
  - earn 85% of points for pre-lecture and in-lecture questions
  - make 85% of the lecture-attendance votes
  - attend 85% of the labs
- If you end up with x% for a given component where x < 85, you will get x/85 of the possible points.
- This policy is designed to allow for occasional absences for special circumstances.
- If you need to miss a lecture:
  - watch its recording ASAP (available on Blackboard)
  - keep up with the pre-lecture tasks and the assignments
  - do not email your instructor!

Course Staff

- **Instructor:** David Sullivan (A1/A2 lectures)
- **Teaching Fellows (TFs)**
  and **Undergrad Course Assistants (CAs)**
  - see the course website for names and photos:
    http://www.cs.bu.edu/courses/cs111/staff.shtml
- Office-hour calendar:
  http://www.cs.bu.edu/courses/cs111/office_hours.shtml
- For questions: **post on Piazza** or cs111-staff@cs.bu.edu

Algorithms

- In order to solve a problem using a computer, you need to come up with one or more *algorithms*.
- An algorithm is a step-by-step description of how to accomplish a task.
- An algorithm must be:
  - *precise:* specified in a clear and unambiguous way
  - *effective:* capable of being carried out

Programming

- Programming involves expressing an algorithm in a form that a computer can interpret.
- We will use the Python programming language.
  - one of many possible languages
  - widely used
  - relatively simple to learn

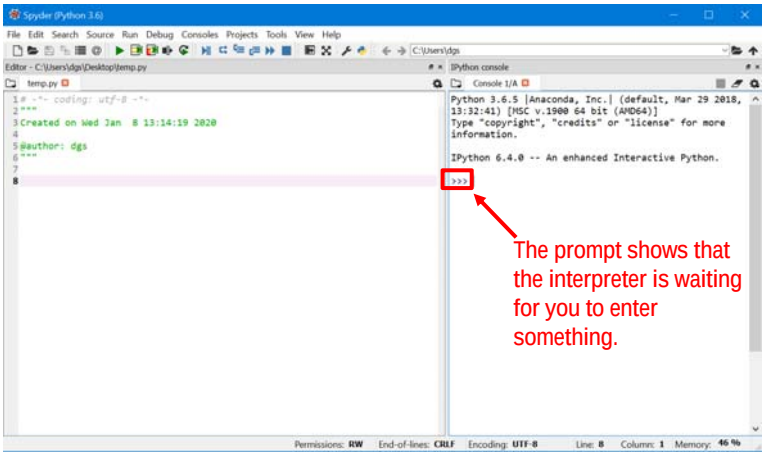
- The key concepts of the course transcend this language.
- See Lab 0 for details on how to install Python.

Pre-Lecture
Getting Started With Python

Computer Science 111
Boston University

Interacting with Python

- We're using Python **3** (*not* 2).
  - see Lab 0 for how to install and configure Spyder
- Two windows in Spyder: the editor and the IPython console



Arithmetic in Python

- Numeric operators include:
  - `+` addition
  - `-` subtraction
  - `*` multiplication
  - `/` division
  - `**` exponentiation
  - `%` modulus: gives the remainder of a division
- Examples:

```
>>> 6 * 7
42
>>> 2 ** 4
16
>>> 17 % 2
1
>>> 17 % 3
_________
```

Arithmetic in Python (cont.)

- The operators follow the standard order of operations.
  - example: multiplication before addition
- You can use parentheses to force a different order.
- Examples:

```
>>> 2 + 3 * 5

_____________

>>> (2 + 3) * 5

_____________
```

Data Types

- Different kinds of values are stored and manipulated differently.
- Python *data types* include:
  - integers
    - example: `451`
  - floating-point numbers
    - numbers that include a decimal
    - example: `3.1416`

Data Types and Operators

- There are really two sets of numeric operators:
  - one for integers (ints)
  - one for floating-point numbers (floats)
- In most cases, the following rules apply:
  - if *at least one* of the operands is a float, the result is a float
  - if *both* of the operands are ints, the result is an int
- One exception: division!
- Examples:

Two Types of Division

- The / operator *always* produces a float result.
  - examples:

```
>>> 5 / 3
1.6666666666666667
>>> 6 / 3
```

Two Types of Division (cont.)

- There is a separate // operator for *integer* division.

```
>>> 6 // 3
2
```

- Integer division *discards* any fractional part of the result:

```
>>> 11 // 5
2
>>> 5 // 3
```

- Note that it does *not* round!

Another Data Type

- A *string* is a sequence of characters/symbols
  - surrounded by single or double quotes
  - examples: `"hello"` `'Picobot'`

Pre-Lecture
Program Building Blocks: Variables, Expressions, Statements

Computer Science 111
Boston University

Variables

- Variables allow us to store a value for later use:

```
>>> temp = 77
>>> (temp - 32) * 5 / 9
25.0
```

Expressions

- *Expressions* produce a value.
  - We *evaluate* them to obtain their value.
- They include:
  - *literals* ("hard-coded" values):

    ```
    3.1416
    'Picobot'
    ```
  - variables

    ```
    temp
    ```
  - combinations of literals, variables, and operators:

    ```
    (temp - 32) * 5 / 9
    ```

Evaluating Expressions with Variables

- When an expression includes variables, they are first replaced with their current value.
- Example:

```
(temp - 32) * 5 / 9
( 77  - 32) * 5 / 9
      45    * 5 / 9
          225   / 9
              25.0
```

Statements

- A *statement* is a command that carries out an action.
- A *program* is a sequence of statements.

```
quarters = 2
dimes = 3
nickels = 1
pennies = 4
cents = quarters*25 + dimes*10 + nickels*5 + pennies
print('you have', cents, 'cents')
```

Assignment Statements

- *Assignment statements* store a value in a variable.

```
temp = 20
```

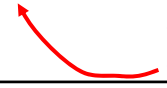
- General syntax:

  variable = expression

= is known as the *assignment operator*

- Steps:
  1) evaluate the expression on the right-hand side of the =
  2) assign the resulting value to the variable on the left-hand side of the =
- Examples:

```
quarters = 10
quarters_val = 25 * quarters
               25 * 10
               250
```



Assignment Statements (cont.)

- We can change the value of a variable by assigning it a new value.
- Example:

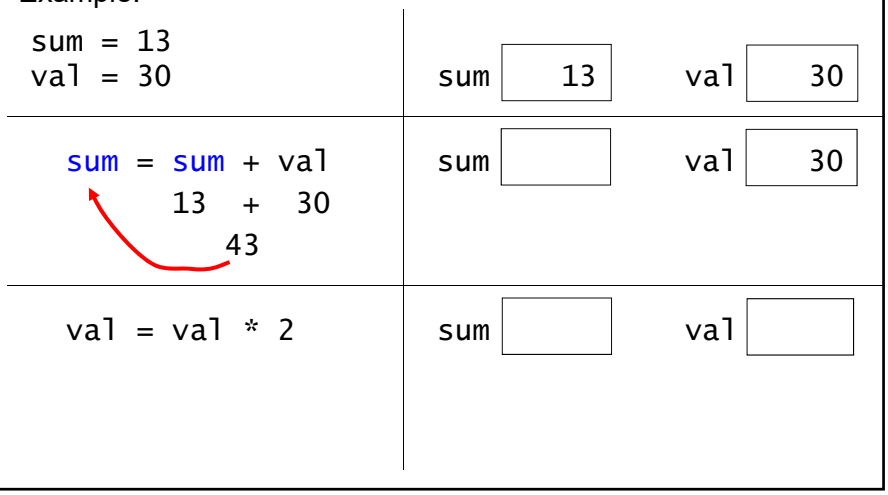
`num1 = 100` `num2 = 120`	num1 100 num2 120
`num1 = 50`	num1 num2 120
`num1 = num2 * 2`	num1 num2 120
`num2 = 60`	num1 num2

Assignment Statements (cont.)

- An assignment statement does not create a permanent relationship between variables.
- ***You can only change the value of a variable by assigning it a new value!***

Assignment Statements (cont.)

- A variable can appear on both sides of the assignment operator!
- Example:



Creating a Reusable Program

- Put the statements in a text file.

```
# a program to compute the value of some coins

quarters = 2       # number of quarters
dimes = 3
nickels = 1
pennies = 4

cents = quarters*25 + dimes*10 + nickels*5 + pennies
print('you have', cents, 'cents')
```

- Program file names should have the extension `.py`
  - example: `coins.py`

Print Statements

- print statements display one or more values on the screen
- Basic syntax:

  `print(`*expr*`)`

  or

  `print(`$expr_1$`, `$expr_2$`, ... `$expr_n$`)`

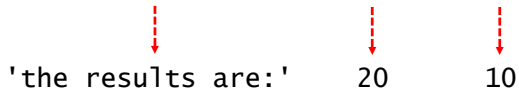
  where each *expr* is an expression
- Steps taken when executed:
  1) the individual expression(s) are evaluated
  2) the resulting values are displayed on the same line, *separated by spaces*
- To print a blank line, omit the expressions:

  ```
  print()
  ```

Print Statements (cont.)

- Examples:
  - first example:

    ```
    print('the results are:', 15 + 5, 15 - 5)
    ```

    output: `the results are: 20 10`

    (note that the quotes around the string literal are *not* printed)
  - second example:

    ```
    cents = 89
    print('you have', cents, 'cents')
    ```

    output: _____

Variables and Data Types

- The `type` function gives us the type of an expression:

```
>>> type('hello')
<class 'str'>
>>> type(5 / 2)
<class 'float'>
```

- Variables in Python do *not* have a fixed type.
  - examples:

```
>>> temp = 25.0
>>> type(temp)
<class 'float'>
>>> temp = 77
>>> type(temp)
```

Python Basics

Computer Science 111
Boston University

What is the output of the following program?

```
x = 15
name = 'Picobot'
x = x // 2
print('name', x, type(x))
```

What about this program?

```
x = 15
name = 'Picobot'
x = 7.5
print(name, 'x', type(x))
```

What are the values of the variables after the following code runs?

```
x = 5
y = 6
x = y + 3
z = x + y
x = x + 2
```

x	y	z
5		
5	6	

Complete this table to keep track of the values of the variables!

What are the values of the variables after the following code runs?

```
x = 5
y = x ** 2
z = x % 3
x + 2
```

x	y	z

On paper, make a table for the values of your variables!

Pre-Lecture
Strings

Computer Science 111
Boston University

Strings: Numbering the Characters

- The position of a character within a string is known as its *index*.
- There are two ways of numbering characters in Python:
  - from left to right, starting from 0

```
 0 1 2 3 4
'Perry'
```

  - from right to left, starting from -1

```
-5-4-3-2-1
'Perry'
```

  - `'P'` has an index of 0 or -5
  - `'y'` has an index of _____

String Operations

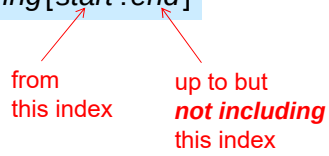
- Indexing: *string*[*index*]

```
>>> name = 'Picobot'
>>> name[1]
'i'
>>> name[-3]
```

- Slicing (extracting a substring): *string*[*start*:*end*]

```
>>> name[0:2]
'Pi'
>>> name[1:-1]
```

```
>>> name[1:]
'icobot'
>>> name[:4]
```



String Operations (cont.)

- Concatenation: *string1* + *string2*

```
>>> word = 'program'
>>> plural = word + 's'
>>> plural
'programs'
```

- Duplication: *string* * *num_copies*

```
>>> 'ho!' * 3
'ho!ho!ho!'
```

- Determining the length: `len`(*string*)

```
>>> name = 'Perry'
>>> len(name)
5
>>> len('')   # an empty string - no characters!
0
```

Skip-Slicing

- Slices can have a third number: *string*[*start*:*end*:*stride_length*]


```
>>> s[0:8:2]
'bso '              # note the space at the end!
```

Skip-Slicing (cont.)

- Slices can have a third number: *string*[*start*:*end*:*stride_length*]


```
>>> s[5:0:-1]
_____________
```

Pre-Lecture
Lists

Computer Science 111
Boston University

Lists

- Recall: A string is a sequence of characters.

```
'hello'
```

- A list is a sequence of *arbitrary* values (the list's *elements*).

```
[2, 4, 6, 8]
['CS', 'math', 'english', 'psych']
```

- A list can include values of different types:

```
['Star Wars', 1977, 'PG', [35.9, 460.9]]
```

List Ops == String Ops (more or less)

```
>>> majors = ['CS', 'math', 'english', 'psych']
>>> majors[2]
'english'
>>> majors[1:3]
_____________
>>> len(majors)
_____________
>>> majors + ['physics']
['CS', 'math', 'english', 'psych', 'physics']
>>> majors[::-2]
_________________
```

Note the difference!

- For a string, both slicing and indexing produce a string:

```
>>> s = 'Terriers'
>>> s[1:2]
'e'
>>> s[1]
'e'
```

- For a list:
  - slicing produces a list
  - indexing produces a single element – may or may not be a list

```
>>> info = ['Star Wars', 1977, 'PG', [35.9, 460.9]]
>>> info[1:2]
[1977]
>>> info[1]
1977
>>> info[-1]
____________________
```

```
>>> info[2:]
____________________
>>> info[-1][-1]
460.9
>>> info[0][-4]
____________________
```

Strings and Lists

Computer Science 111
Boston University

What is the value of s after the following code runs?

```
s = 'abc'

s = ('d' * 3) + s

s = s[2:-2]
```

Fill in the blank to make the code print `'compute!'`

```
subject = 'computer science!'
verb = _________________
print(verb)
```

Skip-Slicing

- Slices can have a third number: *string*[*start* : *end* : *stride_length*]

```
s = 'boston university terriers'
    0  1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
```

```
>>> s[0:8:2]
'bso '            # note the space at the end!

>>> s[5:0:-1]
'notso'

>>> s[  :  :  ]  # what numbers do we need?
'viti'

>>> s[12:21:8] + s[21::3]    # what do we get?
```

What is the output of the following program?

```
mylist = [1, 2, [3, 4, 5]]
print(mylist[1], mylist[1:2])
```

Note the difference!

- For a string, both slicing and indexing produce a string:

```
>>> s = 'Terriers'
>>> s[1:2]
'e'
>>> s[1]
'e'
```

- For a list:
  - slicing produces a list
  - indexing produces a single element – may or may not be a list

```
>>> info = ['Star Wars', 1977, 'PG', [35.9, 460.9]]
>>> info[1:2]
[1977]
>>> info[1]
1977
>>> info[-1]
[35.9, 460.9]
```

```
>>> _____________________
35.9
```

How could you fill in the blank to produce `[105, 111]`?

```
intro_cs = [101, 103, 105, 108, 109, 111]

dgs_courses = _________________________
```

A. `intro_cs[2:3] + intro_cs[-1:]`

B. `intro_cs[-4] + intro_cs[5]`

C. `intro_cs[-4] + intro_cs[-1:]`

D. more than one of the above

E. none of the above

Extra practice from the textbook authors!

```
pi = [3,1,4,1,5,9]
L = [ 'pi', "isn't", [4,2] ]
M = 'You need parentheses for chemistry !'
    0   4   8   12  16  20  24  28  32
```

Part 1

What is `len(pi)`

What is `len(L)`

What is `len(L[1])`

What is `pi[2:4]`

What slice of `pi` is `[3,1,4]`

What slice of `pi` is `[3,4,5]`

Part 2

What is `L[0]`

These two are different!

What is `L[0:1]`

What is `L[0][1]`

What slice of `M` is `'try'`?

is `'shoe'`?

What is `M[9:15]`

What is `M[::5]`

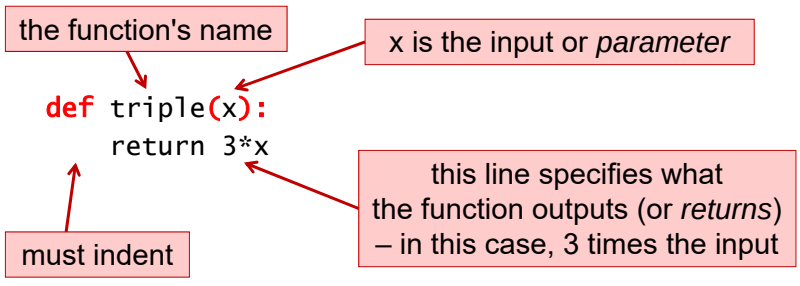
Extra! What are `pi[0]*(pi[1] + pi[2])` and `pi[0]*(pi[1:2] + pi[2:3])` ?

These two are different, too…

Pre-Lecture
Introduction to Functions

Computer Science 111
Boston University

Defining a Function



Multiple Lines, Multiple Parameters

```
def circle_area(diam):
    """ Computes the area of a circle
        with a diameter diam.
    """
    radius = diam / 2
    area = 3.14159 * (radius**2)
    return area

def rect_perim(l, w):
    """ Computes the perimeter of a rectangle
        with length l and width w.
    """
    return 2*l + 2*w
```

What is the output of this code?

```
def calculate(x, y):
    a = y
    b = x + 1
    return a + b + 3


print(calculate(3, 2))
```

x	y	a	b
3	2		

(complete the rest on the next slide)

The values in the function call are assigned to the parameters.

In this case, it's as if we had written:

```
x = 3
y = 2
```

What is the output of this code? (cont.)

x y a b

```
def calculate(x, y):
    a = y
    b = x + 1
    return a + b + 3


print(calculate(3, 2))
```

The output/return value:
- is sent back to where the function call was made
- replaces the function call

The program picks up where it left off when the function call was made.

Intro. to Functions

Computer Science 111
Boston University

Functions With String Inputs

```
def undo(s):
    """ Adds the prefix "un" to the input s. """
    return 'un' + s

def redo(s):
    """ Adds the prefix "re" to the input s. """
    return 're' + s
```

- Examples:

```
>>> undo('plugged')

>>> undo('zipped')

>>> redo('submit')

>>> redo(undo('zipped'))
```

What is the output of this program?

```
def mystery1(t):
    return t[::-1]

def mystery2(t):
    return t[0] + t[-1]

s = 'terriers'
mystery1(s)
print(mystery2(s))
```

A. `ts`

B. `st`

C. `sreirret`
   `ts`

D. `sreirret`
   `st`

What is the output of this code?

```
def calculate(x, y):
    a = y
    b = x + 1
    return a * b - 3


print(calculate(4, 1))
```

x	y	a	b

Practice Writing a Function

- Write a function `avg_first_last(values)` that:
  - takes a list `values` that has at least one element
  - returns the average of the first and last elements
  - examples:

```
>>> avg_first_last([2, 6, 3])
2.5                  # average of 2 and 3 is 2.5
>>> avg_first_last([7, 3, 1, 2, 4, 9])
8.0                  # average of 7 and 9 is 8.0
```

```
def avg_first_last(values):
    first = _____________
    last = _____________
    return _________________
```

Returning vs. Printing

- Our previous function *returns* the result:

```
def avg_first_last(values):
    ...
    return ____________
```

- Would it be equivalent to print the result?

```
def avg_first_last(values):
    ...
    print(____________)
```

- If the function prints instead of returning, you can't do something like this:

```
avg = avg_first_last([5, 7, 9, 10, 12])
print('The result is', avg)
```

More Practice

- Write a function `middle_elem(values)` that:
  - takes a list `values` that has at least one element
  - returns the element in the middle of the list
    - when there are two middle elements, return the one closer to the end
  - examples:

```
>>> middle_elem([2, 6, 3])
6
>>> middle_elem([7, 3, 1, 2, 4, 9])
2
```

```
def middle_elem(values):
    middle_index = _____________
    return _________________
```

Pre-Lecture
Making Decisions: Conditional Execution

Computer Science 111
Boston University

Conditional Execution

- Conditional execution allows your code to *decide* whether to do something, based on some condition.
  - example:

```
def abs_value(x):
    """ returns the absolute value of input x """
    if x < 0:
        x = -1 * x
    return x
```

  - examples of calling this function from the Shell:

```
>>> abs_value(-5)
5
>>> abs_value(10)
```

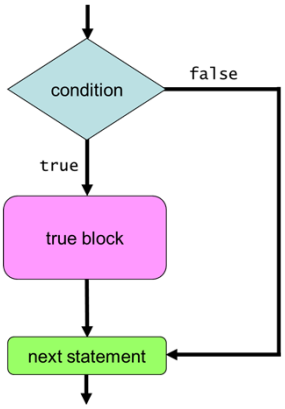
Simple Decisions: `if` Statements

- Syntax:

```
if condition:
    true block
```

where:

- *condition* is an expression that is true or false
- *true block* is one or more indented statements



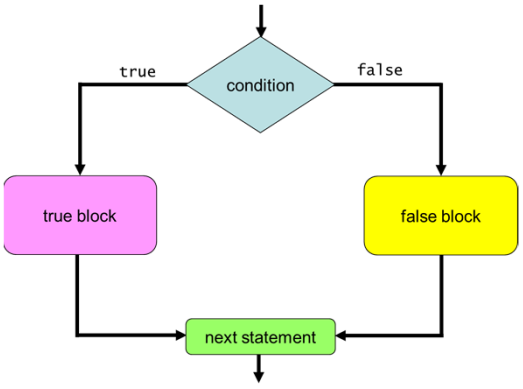
- Example:

```
def abs_value(x):
    if x < 0:
        x = -1 * x    # true block
    return x
```

Two-Way Decisions: `if-else` Statements

- Syntax:

```
if condition:
    true block
else:
    false block
```



- Example:

```
def pass_fail(avg):
    if avg >= 60:
        grade = 'pass'    # true block
    else:
        grade = 'fail'    # false block
    return grade
```

Expressing Simple Conditions

- Python provides a set of *relational operators* for making comparisons:

operator	name	examples
`<`	less than	`val < 10` `price < 10.99`
`>`	greater than	`num > 60` `state > 'Ohio'`
`<=`	less than or equal to	`average <= 85.8`
`>=`	greater than or equal to	`name >= 'Jones'`
`==` *(don't confuse with =)*	equal to	`total == 10` `letter == 'P'`
`!=`	not equal to	`age != my_age`

Boolean Values and Expressions

- A condition has one of two values: `True` or `False`.

```
>>> 10 < 20
True
>>> "Jones" == "Baker"
False
```

- `True` and `False` are *not* strings.
  - they are literals from the `bool` data type

```
>>> type(True)
<class 'bool'>
>>> type(30 > 6)
```

- An expression that evaluates to `True` or `False` is known as a *boolean expression.*

Forming More Complex Conditions

- Python provides *logical operators* for combining/modifying boolean expressions:

name	example and meaning
`and`	`age >= 18 and age <= 35` `True` *if both conditions are* `True`*, and* `False` *otherwise*

`or`	`age < 3 or age > 65`
	`True` *if one or both of the conditions are* `True`*;*
	`False` *if both conditions are* `False`

`not`	`not (grade > 80)` `True` *if the condition is* `False`*, and* `False` *if it is* `True`

A Word About Blocks

- A block can contain multiple statements.

```
def welcome(class):
    if class == 'frosh':
        print('Welcome to BU!')
        print('Have a great four years!')
    else:
        print('Welcome back!')
        print('Have a great semester!')
        print('Be nice to the frosh students.')
```

- A new block *begins* whenever we *increase* the amount of indenting.
- A block *ends* when we either:
  - reach a line with *less* indenting than the start of the block
  - reach the end of the program

Multi-Way Decisions

- The following function doesn't work.

avg grade

```
def letter_grade(avg):
    if avg >= 90:
        grade = 'A'
    if avg >= 80:
        grade = 'B'
    if avg >= 70:
        grade = 'C'
    if avg >= 60:
        grade = 'D'
    else:
        grade = 'F'
    return grade
```

- example:

```
>>> letter_grade(95)
_______________
```

Multi-Way Decisions (cont.)

- Here's a fixed version:

avg grade

```
def letter_grade(avg):
    if avg >= 90:
        grade = 'A'
    elif avg >= 80:
        grade = 'B'
    elif avg >= 70:
        grade = 'C'
    elif avg >= 60:
        grade = 'D'
    else:
        grade = 'F'
    return grade
```

- example:

```
>>> letter_grade(95)
_______________
```

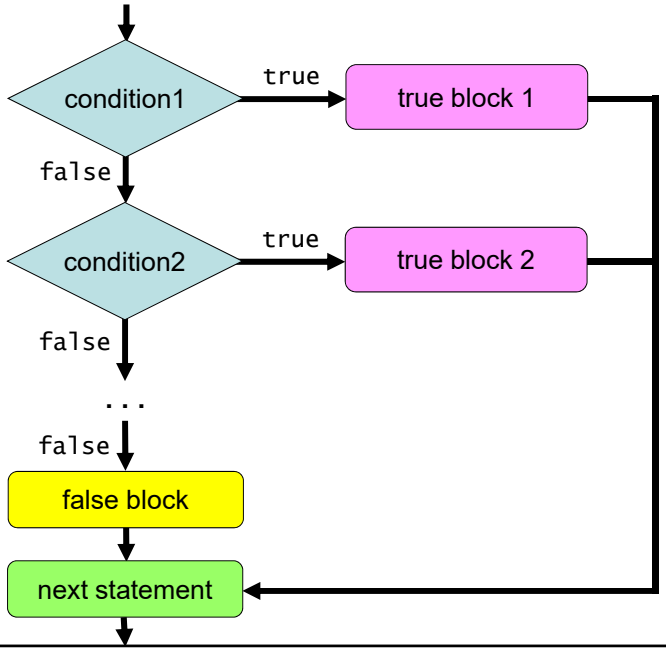
Multi-Way Decisions: `if-elif-else` Statements

- Syntax:

```
if condition1:
    true block for condition1
elif condition2:
    true block for condition2
elif condition3:
    true block for condition3
...
else:
    false block
```

- The conditions are evaluated in order. The true block of the *first* true condition is executed.
- If none of the conditions are true, the false block is executed.

Flowchart for an `if-elif-else` Statement



Making Decisions: Conditional Execution

Computer Science 111
Boston University

Making Decisions

- **One-way:** deciding whether or not to do something

```
if x < 0:
    print('x is negative')
    x = -1 * x
```

- **Two-way:** choosing among two options

```
if x < 0:
    print('x is negative')
    x = -1 * x
else:
    print('x is non-negative')
```

Recall: A Word About Blocks

- A block can contain multiple statements.

```
def welcome(class):
    if class == 'frosh':
        print('Welcome to BU!')
        print('Have a great four years!')
    else:
        print('Welcome back!')
        print('Have a great semester!')
        print('Be nice to the frosh students.')
```

- A new block *begins* whenever we *increase* the amount of indenting.
- A block *ends* when we either:
  - reach a line with *less* indenting than the start of the block
  - reach the end of the program

Nesting

- We can "nest" one conditional statement in the true block or false block of another conditional statement.

```
def welcome(class):
    if class == 'frosh':
        print('Welcome to BU!')
        print('Have a great four years!')
    else:
        print('Welcome back!')
        if class == 'senior':
            print('Have a great last year!')
        else:
            print('Have a great semester!')
        print('Be nice to the frosh students.')
```

What is the output of this program?

```
x = 5
if x < 15:
    if x > 8:
        print('one')
    else:
        print('two')
else:
    if x > 2:
        print('three')
```

What does this print? (note the changes!)

```
x = 5
if x < 15:
    if x > 8:
        print('one')
    else:
        print('two')
if x > 2:
    print('three')
```

What does this print? (note the new changes!)

```
x = 5
if x < 15:
    if x > 8:
        print('one')
else:
    print('two')
if x > 2:
    print('three')
```

How many lines does this print?

```
x = 5
if x == 8:
    print('how')
elif x > 1:
    print('now')
elif x < 20:
    print('wow')
print('cow')
```

How many lines does this print?

```
x = 5
if x == 8:
    print('how')
if x > 1:
    print('now')
if x < 20:
    print('wow')
print('cow')
```

What is the output of this code?

```
def mystery(a, b):
    if a == 0 or a == 1:
        return b
    return a * b


print(mystery(0, 5))
```

Common Mistake When Using and / or

```
def mystery(a, b):
    if a == 0 or 1:      # this is problematic
        return b
    return a * b

print(mystery(0, 5))
```

- When using and / or, both sides of the operator should be a boolean expression that could stand on its own.

boolean	*boolean*	*boolean*	*integer*
`a == 0 or a == 1`		`a == 0 or 1`	
(do this)		*(don't do this)*	

- Unfortunately, Python *doesn't* complain about code like the problematic code above.
  - but it won't typically work the way you want it to!

Avoid Overly Complicated Code

- The following also involves decisions based on a person's age:

```
age = ...   # let the user enter his/her age
if age < 13:
    print('You are a child.')
elif age >= 13 and age < 20:
    print('You are a teenager.')
elif age >= 20 and age < 30:
    print('You are in your twenties.')
elif age >= 30 and age < 40:
    print('You are in your thirties.')
else:
    print('You are really old.')
```

- How could it be simplified?

Pre-Lecture
Variable Scope

Computer Science 111
Boston University

Local Variables

```
def mystery(x, y):
    b = x - y        # b is a local var of mystery
    return 2*b       # we can access b here

c = 7
mystery(5, 2)
print(b + c)         # we can't access b here
```

- When we assign a value to a variable inside a function, we create a *local variable*.
  - it "belongs" to that function
  - it can't be accessed outside of that function
- The parameters of a function are also limited to that function.
  - example: the parameters x and y above

Global Variables

```
def mystery(x, y):
    b = x - y
    return 2*b + c    # works, but not recommended

c = 7                 # c is a global variable
mystery(5, 2)
print(b + c)          # we can access c here
```

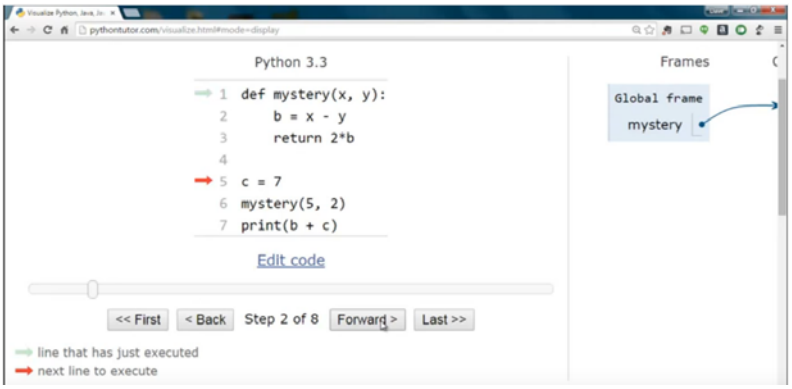
- When we assign a value to a variable *outside* of a function, we create a *global variable*.
  - it belongs to the *global scope*
- A global variable can be used anywhere in your program.
  - in code that is outside of any function
  - in code inside a function (but this is not recommended!)

Different Variables With the Same Name!

```
def mystery(x, y):
    b = x - y         # this b is local
    return 2*b        # we access the local b here

b = 1                 # this b is global
c = 7
mystery(5, 2)
print(b + c)          # we access the global b here
```

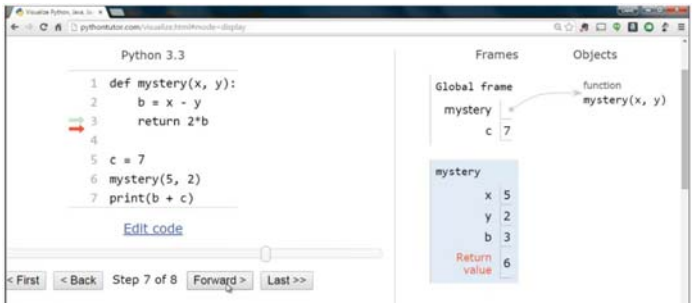
- The program above has two different variables called b.
  - one local variable
  - one global variable
- When this happens, the *local* variable has priority inside the function to which it belongs.

Python Tutor



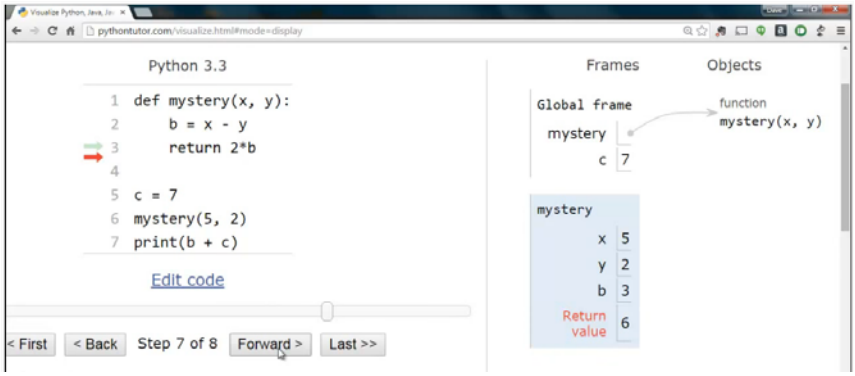
- Python Tutor allows us to trace through a program's execution.
  - use the *Forward* button
- The red arrow shows the next line to execute.
- The pale arrow shows the line that was just executed.

Frames for Variables



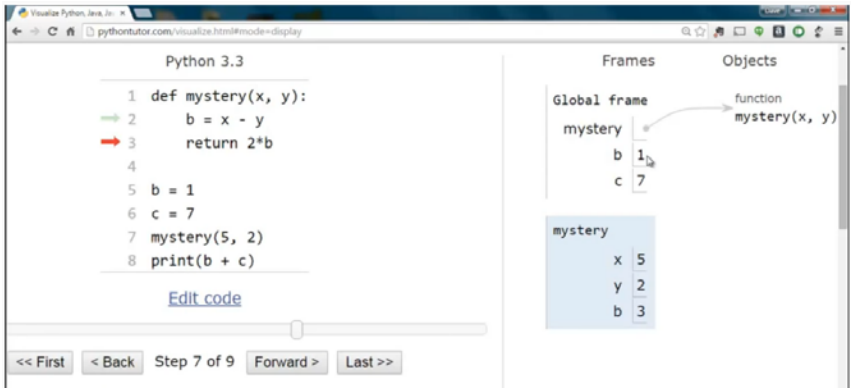
- Variables are stored in blocks of memory known as *frames.*
  - stored in a region of memory known as the *stack*
- Global variables are stored in the *global frame*.
- Each function call gets a frame for its local variables.
  - goes away when the function returns

Frames for Variables (cont.)



- Where is the error is this program?

Frames for Variables (cont.)



- What is the output of this fixed version of the program?

Pre-Lecture
Functions Calling Functions

Computer Science 111
Boston University

Finding the Distance Between Two Points

$$d=\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

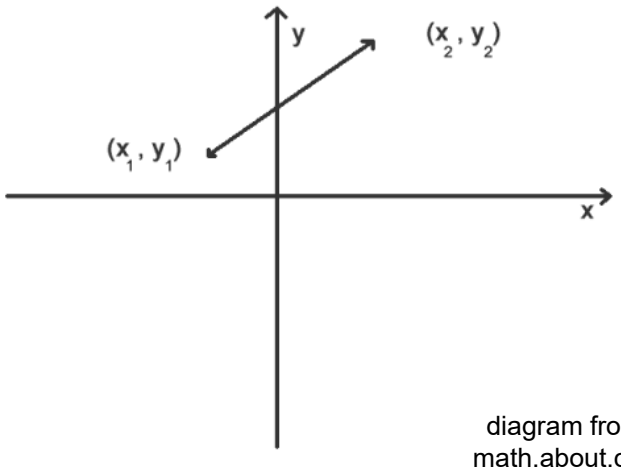


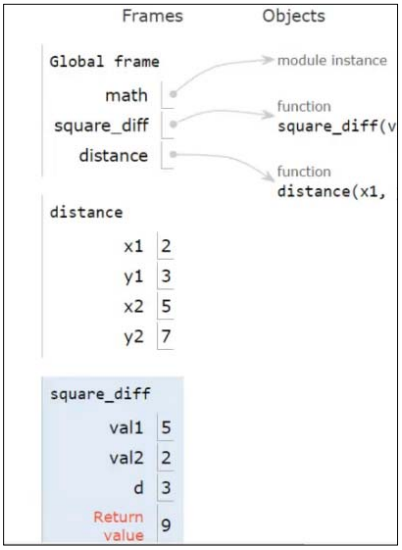
diagram from:
math.about.com

A Program for Computing Distance

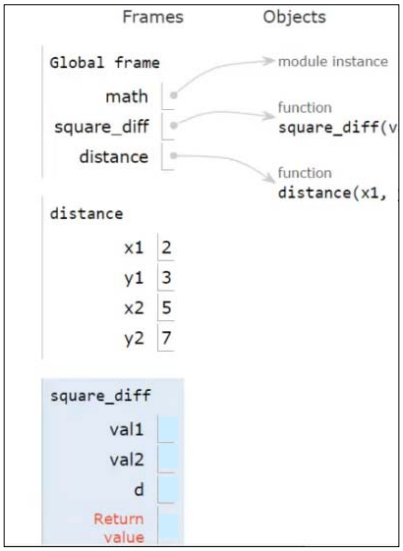
```
import math

def square_diff(val1, val2):
    """ returns the square of val1 - val2 """
    d = val1 - val2
    return d ** 2

def distance(x1, y1, x2, y2):
    """ returns the distance between two points
        in a plane with coordinates (x1, y1)
        and (x2, y2)
    """
    d = square_diff(x2, x1) + square_diff(y2, y1)
    dist = math.sqrt(d)
    return dist

dist = distance(2, 3, 5, 7)
print('distance between (2, 3) and (5, 7) is', dist)
```

Tracing the Program in Python Tutor

- stack frames during the 1st call to `square_diff`:



- fill in the stack frame for the 2nd call:



Variable Scope
Functions Calling Functions

Computer Science 111
Boston University

What is the output of this code?

```
def mystery2(a, b):
    x = a + b
    return x + 1

x = 8
mystery2(3, 2)
print(x)
```

What is the output of this code? (version 2)

```
def mystery2(a, b):
    x = a + b
    return x + 1

x = 8
mystery2(3, 2)
print(x)
```

A Note About Globals

- It's not a good idea to access a global variable inside a function.
  - for example, you shouldn't do this:

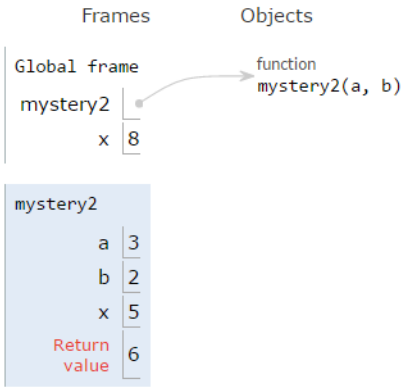
```
def average3(a, b):
    total = a + b + c   # accessing a global c
    return total/3

c = 8
print(average3(5, 7))
```

- Instead, you should pass it in as a parameter/input:

```
def average3(a, b, c):
    total = a + b + c   # accessing input c
    return total/3

c = 8
print(average3(5, 7, c))
```

Recall: Frames and the Stack

- Variables are stored in blocks of memory known as *frames.*
- Each function call gets a frame for its local variables.
  - goes away when the function returns
- Global variables are stored in the global frame.
- The *stack* is the region of the computer's memory in which the frames are stored.
  - thus, they are also known as *stack frames*



What is the output of this code?

```
def quadruple(y):
    y = 4 * y
    return y

y = 8
quadruple(y)

print(y)
```

How could we change this to see the return value?

```
def quadruple(y):
    y = 4 * y
    return y

y = 8
quadruple(y)

print(y)
```

What is the output of this program?

```
def demo(x):
   return x + f(x)

def f(x):
   return 11*g(x) + g(x//2)

def g(x):
   return -1 * x

print(demo(-4))
```

demo
```
x = -4
return -4 + f(-4)
```

f
```
x = -4
return 11*g(-4) + g(-4//2)
```

g frame for 1st call
```
x = -4
return -1 * x
```

g frame for 2nd call
```
x =
return -1 * x
```

Tracing Function Calls

```
def foo(x, y):
    y = y + 1
    x = x + y
    print(x, y)
    return x

x = 2
y = 0

y = foo(y, x)
print(x, y)

foo(x, x)
print(x, y)

print(foo(x, y))
print(x, y)
```

foo
x | y

global
x | y

output

Full Trace of First Example

```
def quadruple(y):     # 3. local y = 8
    y = 4 * y         # 4. local y = 4 * 8 = 32
    return y          # 5. return local y's value
           32
y = 8             # 1. global y = 8
quadruple(y)      # 2. pass in global y's value
                  # 6. return value is thrown away!
print(y)          # 7. print global y's value,
                  #    which is unchanged!
```

You ***can't*** change the value of a variable by passing it into a function!

Pre-Lecture
A First Look at Recursion

Computer Science 111
Boston University

Functions Calling Themselves: *Recursion!*

```
def fac(n):
    if n <= 1:
        return 1
    else:
        return n * fac(n - 1)
```

- Recursion solves a problem by reducing it to a *simpler* or *smaller* problem *of the same kind*.
  - the function calls itself to solve the smaller problem!
- We take advantage of *recursive substructure*.
  - the fact that we can define the problem *in terms of itself*

    `n! = n * (n-1)!`

Functions Calling Themselves: *Recursion!* (cont.)

```
def fac(n):
    if n <= 1:
        return 1
    else:
        return n * fac(n - 1)
```

base case

recursive case

- One recursive call leads to another...

```
fac(5) = 5 * fac(4)
       = 5 * 4 * fac(3)
       = ...
```

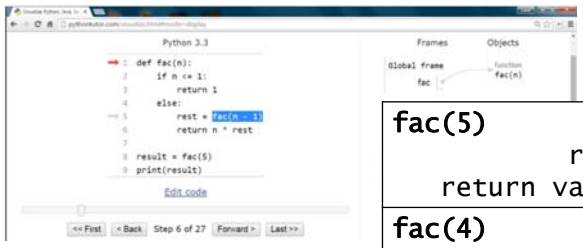
- We eventually reach a problem that is small enough to be solved directly – a *base case.*
  - stops the recursion
  - make sure that you always include one!

Alternative Version of `fac(n)`

```
def fac(n):
    if n <= 1:
        return 1

    else:
        rest = fac(n - 1)
        return n * rest
```

- Many people find this easier to read/write/understand.
- Storing the result of the recursive call will occasionally make the problem easier to solve.
- It also makes your recursive functions easier to trace and debug.
- ***We highly recommend that you take this approach!***

Tracing Recursion in Python Tutor



Fill in the stack frames!

`fac(5)` n: rest: return value:
`fac(4)` n: rest: return value:
`fac(3)` n: rest: return value:
`fac(2)` n: rest: return value:
`fac(1)` n: rest: return value:

Pre-Lecture
Using Recursion, Part I

Computer Science 111
Boston University

Recursively Processing a List or String

- Sequences are recursive!
  - a string is a character followed by a string...
  - a list is an element followed by a list...
- Let `s` be the sequence (string or list) that we're processing.
- Do one step!
  - use `s[0]` to access the initial element
  - do something with it
- Delegate the rest!
  - use `s[1:]` to get the rest of the sequence.
  - make a recursive call to process it!

Recursively Finding the Length of a String

```
def mylen(s):
    """ returns the number of characters in s
        input s: an arbitrary string
    """
    if                      # base case


    else:
```

- Ask yourself:

(base case) When can I determine the length of s *without* looking at a smaller string?

(recursive substructure) How could I use the length of ***anything smaller*** than s to determine the length of s?

How recursion works...

```
mylen('wow')
  s = 'wow'
  len_rest = mylen('ow')
  return
```

```
mylen('ow')
  s =
  len_rest =
  return
```

4 different stack frames, each with its own s and len_rest

The final result gets built up *on the way back* from the base case!

Recursively Raising a Number to a Power

```
def power(b, p):
    """ returns b raised to the p power
        inputs: b is a number (int or float)
                p is a non-negative integer
    """
    if                      # base case


    else:
```

- Ask yourself:

(base case) When can I determine b^p *without* determining a smaller power?

(recursive substructure) How could I use ***anything smaller*** than b^p to determine b^p?

How recursion works...

```
power(3, 3)
  b = 3, p = 3
  pow_rest = power(3, 2)
  return
```

```
power(3, 2)
  b = 3, p = 2
  pow_rest =
  return
```

4 different stack frames, each with its own `b`, `p` and `pow_rest`

The final result gets built up *on the way back* from the base case!

A First Look at Recursion

Computer Science 111
Boston University

Recall: Functions Calling Themselves: *Recursion!*

```
def fac(n):
    if n <= 1:
        return 1
    else:
        fac_rest = fac(n - 1)
        return n * fac_rest
```

base case

recursive case

- One recursive call leads to another...
- We eventually reach a problem that is small enough to be solved directly – a *base case.*
  - stops the recursion
  - make sure that you always include one!

Let Recursion Do the Work For You!

```
def fac(n):
    if n <= 1:
        return 1
    else:
        fac_rest = fac(n-1)
        return n * fac_rest
```

You handle the base case – the easiest case!

Recursion does almost all of the rest of the problem!

You specify one step at the end.

How many times will `mylen()` be called?

```
def mylen(s):
    if s == '':             # base case
        return 0
    else:                   # recursive case
        len_rest = mylen(s[1:])
        return len_rest + 1

print(mylen('step'))
```

```
def mylen(s):
    if s == '':
        return 0
    else:
        len_rest = mylen(s[1:])
        return len_rest + 1
```

```
mylen('step')
  s = 'step'
  len_rest = mylen('tep')
```

```
mylen('tep')
  s = 'tep'
  len_rest = mylen('ep')
```

```
mylen('ep')
  s = 'ep'
  len_rest = mylen('p')
```

Fill in the rest of the stack frames!

What is the output of this program?

```
def foo(x, y):
  if x <= y:
     return y
  else:
     return x + foo(x-2,y+1)

print(foo(9, 2))
```

Fill in the stack frames!
(use as many as you need)

foo(9, 2)	x: y:
foo( )	x: y:
	x: y:
	x: y:
	x: y:

More Recursion!

Computer Science 111
Boston University

Designing a Recursive Function

1. Start by programming the base case(s).
   - *What instance(s) of this problem can I solve directly (without looking at anything smaller)?*
2. Find the recursive substructure.
   - *How could I use the solution to **any smaller version** of the problem to solve the overall problem?*
3. Solve the smaller problem using a recursive call!
   - store its result in a variable
4. Do your one step.
   - build your solution from the result of the recursive call
   - use *concrete cases* to figure out what you need to do

A Recursive Function for Counting Vowels

```
def num_vowels(s):
    """ returns the number of vowels in s
        input s: a string of lowercase letters
    """
    # We'll design this together!
```

- Examples of how it should work:

```
>>> num_vowels('compute')
3
>>> num_vowels('now')
1
```

- The `in` operator will be helpful:

```
>>> 'fun' in 'function'
True
>>> 'i' in 'team'
False
```

Design Questions for `num_vowels()`

(base case) When can I determine the # of vowels in `s` *without* looking at a smaller string?

(recursive substructure) How could I use the solution to ***anything smaller*** than `s` to determine the solution to `s`?

total # of vowels =

total # of vowels =

How Many Lines of This Function Have a Bug?

```
def num_vowels(s):
    if s == '':
        return 0
    else:
        num_rest = num_vowels(s[0:])
        if s[0] in 'aeiou':
            return 1
        else:
            return 0
```

After you make your group vote, fix the function!

What value is eventually assigned to num_rest? (i.e., what does the recursive call return?)

```
def num_vowels(s):
    if s == '':
        return 0
    else:
        num_rest = num_vowels(_________________)
        ...


num_vowels('aha')
```

num_vowels('aha')
  s = 'aha'
  num_rest = ??

How recursion works...

```
num_vowels('aha')
  s = 'aha'
  num_rest = num_vowels('ha')
```

```
num_vowels(      )
  s =
  num_rest =
```

Debugging Technique: Adding Temporary prints

```
def num_vowels(s):
    print('beginning call for', s)
    if s == '':
        print('base case returns 0')
        return 0
    else:
        num_rest = num_vowels(s[1:])
        if s[0] in 'aeiou':
            print('call for', s, 'returns', 1 + num_rest)
            return 1 + num_rest
        else:
            print('call for', s, 'returns', 0 + num_rest)
            return 0 + num_rest
```

Pre-Lecture
Using Recursion, Part II

Computer Science 111
Boston University

Recursively Finding the Largest Element in a List

- `mymax(values)`
  - input: a *non-empty* list
  - returns: the largest element in the list
- examples:

```
>>> mymax([5, 8, 10, 2])
10
>>> mymax([30, 2, 18])
30
```

Design Questions for `mymax()`

(base case) When can I determine the largest element in a list without needing to look at a smaller list?

(recursive case) How could I use the largest element in a smaller list to determine the largest element in the entire list?

`list1 = [30, ]` — *largest element* = 18

`list2 = [5, ]` — *largest element* = 10

`mymax(list1)` → _____

`mymax(list2)` → _____

1. compare the first element to largest element in the rest of the list
2. return the larger of the two

Let the recursive call handle the rest of the list!

Recursively Finding the Largest Element in a List

```
def mymax(values):
    """ returns the largest element in a list
        input: values is a *non-empty* list
    """
    if                          # base case


    else:                       # recursive case
```

Tracing Recursion in Python Tutor

Fill in the stack frames!

```
mymax([10, 12, 5, 8])
       values: [10, 12, 5, 8]
  max_in_rest:
 return value:
```

```
mymax([12, 5, 8])
       values: [12, 5, 8]
  max_in_rest:
 return value:
```

```
mymax(                )
       values:
  max_in_rest:
 return value:
```

```
mymax(                )
       values:
  max_in_rest:
 return value:
```

Practicing Recursive Design

Computer Science 111
Boston University

Recall: Recursively Finding the Largest Element in a List

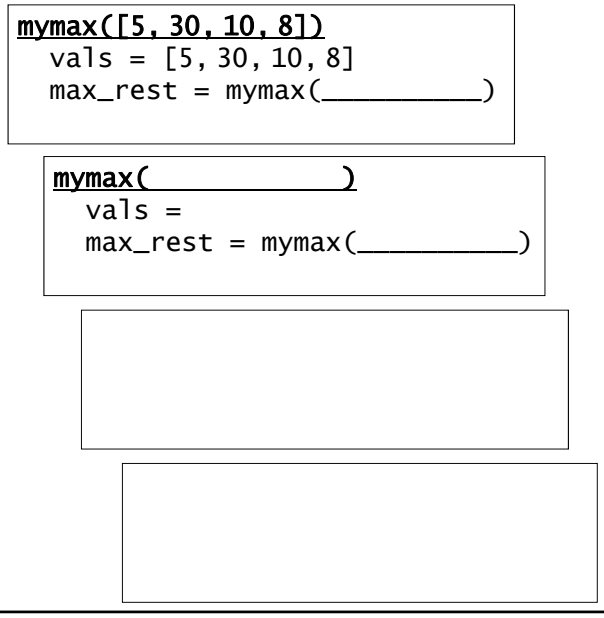
- `mymax(vals)`
  - input: a *non-empty* list
  - returns: the largest element in the list
- examples:

```
>>> mymax([5, 8, 10, 2])
result: 10
>>> mymax([30, 2, 18])
result: 30
```

How many times will max_rest be returned?

```
def mymax(vals):
    if len(vals) == 1:          # base case
        return vals[0]
    else:                       # recursive case
        max_rest = mymax(vals[1:])
        if vals[0] > max_rest:
            return vals[0]
        else:
            return max_rest     # how many times?

print(mymax([5, 30, 10, 8]))
```

How recursion works...



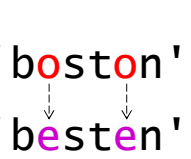
Recall: Designing a Recursive Function

1. Start by programming the base case(s).
   - *What instance(s) of this problem can I solve directly (without looking at anything smaller)?*
2. Find the recursive substructure.
   - *How could I use the solution to **any smaller version** of the problem to solve the overall problem?*
3. Solve the smaller problem using a recursive call!
   - store its result in a variable
4. Do your one step.
   - build your solution on the result of the recursive call
   - use *concrete cases* to figure out what you need to do

Recursively Replacing Characters in a String

- `replace(s, old, new)`
  - inputs: a string `s`
    two characters, `old` and `new`
  - returns: a version of `s` in which all occurrences of `old` are replaced by `new`
- examples:

```
>>> replace('boston', 'o', 'e')
result: 'besten'
```




```
>>> replace('banana', 'a', 'o')
result: 'bonono'

>>> replace('mama', 'm', 'd')

result: ____________
```

Design Questions for `replace()`

(base case) When can I determine the "replaced" version of s *without* looking at a smaller string?

(recursive case) How could I use the "replaced" version of a smaller string to get the "replaced" version of s?

`s1 = 'a____'`

`replace(s1, 'a', 'o')`
=

`s2 = 'r_____'`

`replace(s2, 'e', 'i')`
=

Complete This Function Together!

```
def replace(s, old, new):
    if s == '':
        return _____
    else:
        # recursive call handles rest of string
        repl_rest = replace(______, old, new)
        # do your one step!
        if _________________:
            return _________________
        else:
            return _________________
```

Use our concrete cases!

`replace('always', 'a', 'o')`
`return 'o' + soln to rest of string`

`replace('recurse!', 'e', 'i')`
`return 'r' + soln to rest of string`

More Recursive Design!

Computer Science 111
Boston University

Removing Vowels From a String

- `remove_vowels(s)` - removes the vowels from the string `s`, returning its "vowel-less" version!

```
>>> remove_vowels('recursive')
'rcrsv'
>>> remove_vowels('vowel')
'vwl'
```

- Can we take the usual approach to recursive string processing?
  - base case: empty string
  - delegate `s[1:]` to the recursive call
  - we're responsible for handling `s[0]`

How should we fill in the blanks?

```
def remove_vowels(s):
    if s == '':       # base case
        return _________
    else:             # recursive case
        rem_rest = ___________________

        # do our one step!
        ...
```

Consider this original call…

```
def remove_vowels(s):
    if s == '':
        return ________
    else:
        rem_rest = ______________________

        # do our one step!
        ...
remove_vowels('recurse')
```

What value is eventually assigned to `rem_rest`? (i.e., what does the recursive call return?)

```
def remove_vowels(s):
    if s == '':
        return ________
    else:
        rem_rest = ________________________

        # do our one step!
        ...
remove_vowels('recurse')
```

```
remove_vowels('recurse')
  s = 'recurse'
  rem_rest = ??
```

What should happen after the recursive call?

```
def remove_vowels(s):
    if s == '':
        return ''
    else:
        rem_rest = remove_vowels(s[1:])

        # do our one step!
```

- In our one step, we take care of `s[0]`.
  - we build the solution to the larger problem on the solution to the smaller problem (in this case, `rem_rest`)
  - does what we do depend on the value of `s[0]`?

Consider Concrete Cases

```
remove_vowels('after')     # s[0] is a vowel
```

- what is its solution?
- what is the next smaller subproblem?
- what is the solution to that subproblem?
- how can we use the solution to the subproblem?
  What is our one step?

```
remove_vowels('recurse')   # s[0] is not a vowel
```

- what is its solution? `'`
- what is the next smaller subproblem?
- what is the solution to that subproblem?
- how can we use the solution to the subproblem?
  What is our one step?

remove_vowels()

```
def remove_vowels(s):
    """ returns the "vowel-less" version of s
        input s: an arbitrary string
    """
    if s == '':
        return ''
    else:
        rem_rest = remove_vowels(s[1:])

        # do our one step!
        if s[0] in 'aeiou':

            return ________________

        else:

            return ________________
```

More Recursive Design! `rem_all()`

- `rem_all(elem, values)`
  - inputs: an arbitrary value (`elem`) and a list (`values`)
  - returns: a version of `values` in which *all* occurrences of `elem` in `values` (if any) are removed

```
>>> rem_all(10, [3, 5, 10, 7, 10])
[3, 5, 7]
```

More Recursive Design! `rem_all()`

- Can we take the usual approach to processing a list recursively?
  - base case: empty list
  - delegate `values[1:]` to the recursive call
  - we're responsible for handling `values[0]`
- What are the possible cases for our part (`values[0]`)?
  - does what we do with our part depend on its value?

Consider Concrete Cases

`rem_all(10, [3, 5, 10, 7, 10])` # first value is *not* a match

- what is its solution?
- what is the next smaller subproblem?
- what is the solution to that subproblem?
- how can we use the solution to the subproblem...?
  What is our one step?

`rem_all(10, [10, 3, 5, 10, 7])` # first value *is* a match

- what is its solution?
- what is the next smaller subproblem?
- what is the solution to that subproblem?
- how can we use the solution to the subproblem...?
  What is our one step?

rem_all()

```
def rem_all(elem, values):
    """ removes all occurrences of elem from values
    """
    if values == []:
        return _______
    else:
        rem_rest = rem_all(______, ________)

        if _________________:
            return _________________
        else:
            return _________________
```

Pre-Lecture
List Comprehensions

Computer Science 111
Boston University

Generating a Range of Integers

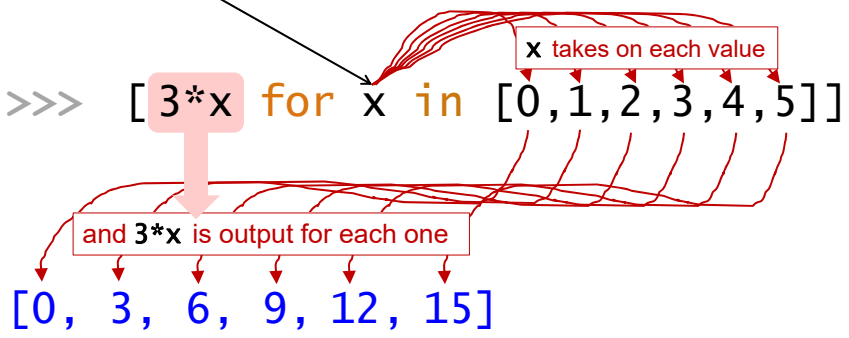
- `range(low, high)`: allows us to work with the range of integers from `low` to `high-1`
  - to see the result produced by `range()` use the `list()` function
  - if you omit `low`, the range will start at 0
- Examples:

```
>>> list(range(3, 10))
[3, 4, 5, 6, 7, 8, 9]

>>> list(range(20, 30))
[20, 21, 22, 23, 24, 25, 26, 27, 28, 29]

>>> list(range(8))
```

List Comprehensions

```
[expression for variable in sequence]
```

this *"runner"* variable can have ***any*** name...



Examples of LCs

```
                        [1, 2, 3, 4]
>>> [111*n for n in range(1, 5)]
[111, 222, 333, 444]

>>> [s[0] for s in ['python', 'is', 'fun!']]

_____________________
```

List Comprehensions (LCs)

- Syntax:

```
[expression for variable in sequence]
                or
[expression for variable in sequence  if boolean]
```

- Examples:

```
                [0, 1, 2, 3, 4, 5]
>>> [2*x for x in range(6) if x % 2 == 0]
[0, 4, 8]

>>> [y for y in ['how', 'now', 'brown'] if len(y) == 3]
```

List Comprehensions

Computer Science 111
Boston University

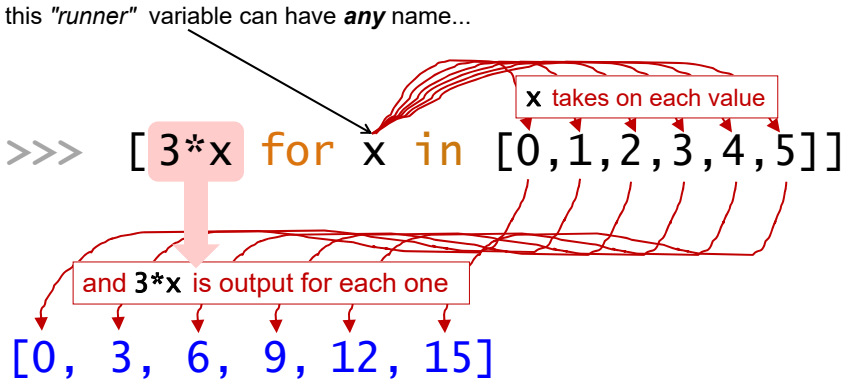
Another Useful Built-In Function

- `sum(list)`: computes & returns the sum of a list of numbers

```
>>> sum([4, 10, 2])
16
```

Recall: List Comprehensions

```
[expression for variable in sequence]
```



More Examples

```
>>> [n - 2 for n in range(10, 15)]

>>> [s[-1]*2 for s in ['go', 'terriers!']]

>>> [c + 'x' for c in 'boston']

>>> [z for z in range(6) if z % 2 == 1]

>>> [z % 4 == 0 for z in [4, 5, 6, 7, 8]]

>>> [1 for x in [4, 5, 6, 7, 8] if x % 4 == 0]

>>> sum([1 for x in [4, 5, 6, 7, 8] if x % 4 == 0])
```

What is the output of this code?

```
lc = [x for x in range(5) if x**2 > 4]

print(lc)
```

LC Puzzles! – Fill in the blanks

```
>>> [__________ for x in range(4)]
[0, 14, 28, 42]


>>> [_________ for s in ['boston', 'university', 'cs']]
['bos', 'uni', 'cs']


>>> [__________ for c in 'compsci']
['cc', 'oo', 'mm', 'pp', 'ss', 'cc', 'ii']


>>> [__________ for x in range(20, 30) if _____________]
[20, 22, 24, 26, 28]


>>> [__________ for w in ['I', 'like', 'ice', 'cream']]
[1, 4, 3, 5]
```

LCs vs. Raw Recursion

```
# raw recursion
def mylen(seq):
    if seq == '' or seq == []:
        return 0
    else:
        len_rest = mylen(seq[1:])
        return 1 + len_rest

# using an LC
def mylen(seq):
    lc = [1 for x in seq]
    return sum(lc)

# here's a one-liner!
def mylen(seq):
    return sum([1 for x in seq])
```

LCs vs. Raw Recursion (cont.)

```
# raw recursion
def num_vowels(s):
    if s == '':
        return 0
    else:
        num_in_rest = num_vowels(s[1:])
        if s[0] in 'aeiou':
            return 1 + num_in_rest
        else:
            return 0 + num_in_rest

# using an LC
def num_vowels(s):
    lc = [1 for c in s if c in 'aeiou']
    return sum(lc)

# here's a one-liner!
def num_vowels(s):
    return sum([1 for c in s if c in 'aeiou'])
```

What list comprehension(s) would work here?

```
def num_odds(values):
    """ returns the number of odd #s in a list
        input: a list of 0 or more integers
    """
    lc = ________________________________________
    return sum(lc)
```

Fill in the Blanks

```
def avg_len(wordlist):
    """ returns the average length of the strings
        in wordlist as a float
        input: a list of 1 or more strings
    """
    lc = [_________ for ____ in __________]

    return ________ / _________
```

```
                      12          6
>>> avg_len(['commonwealth', 'avenue'])
9.0
                 4       4      3      4     2
>>> avg_len(['keep','calm','and','code','on'])
3.4
```

Pre-Lecture
`max()`, `min()`, and Lists of Lists

Computer Science 111
Boston University

`max()` and `min()`

- `max(values)`: returns the largest value in a list of values

```
>>> max([4, 10, 2])
10
>>> max(['all', 'students', 'love', 'recursion'])
'students'
```

- `min(values)`: returns the smallest value in a list of values

```
>>> min([4, 10, 2])
2
>>> min(['all', 'students', 'love', 'recursion'])
```

Lists of Lists

- Recall that the elements of a list can themselves be lists:

```
[[124, 'Jaws'], [150, 'Lincoln'], [115, 'E.T.']]
```

- When you apply `max()`/`min()` to a list of lists, the comparisons are based on the ***first*** element of each sublist:

```
>>> max([[124, 'Jaws'], [150, 'Lincoln'], [115, 'E.T.']])
[150, 'Lincoln']

>>> min([[124, 'Jaws'], [150, 'Lincoln'], [115, 'E.T.']])
```

Problem Solving Using LCs and Lists of Lists

- Sample problem: finding the ***shortest*** word in a list of words.

```
words = ['always', 'come', 'to', 'class']
```

1. Use a list comprehension to build a list of lists:

```
scored_words = [[len(w), w] for w in words]
# for the above words, we get:
```

2. Use min/max to find the correct sublist:

```
min_pair = min(scored_words)
# for the above words, we get:
```

3. Use indexing to extract the desired value from the sublist:

```
min_pair[1]
```